Docker & Kubernetes for Developers

Oturum 1: Kubernetes'e Giriş ve Temel Kavramlar

Eğitmen: Hüseyin Babal
Süre: 2 Saat (Sadece Teori)

Container Orkestrasyon İhtiyacı

Container Dünyasında Yaşanan Zorluklar

- **Manuel Container Yönetimi** → Binlerce container nasıl yönetilecek?
- **Ölçeklenebilirlik** → Trafik artışında otomatik scaling
- **Yüksek Erişilebilirlik** → Container crash olduğunda ne yapılacak?
- **Load Balancing** → Trafiği container'lar arasında dağıtma
- **Service Discovery** → Microservices birbirini nasıl bulacak?
- **Configuration Management** → Farklı ortamlarda config yönetimi

🏭 Modül 1.1: Container Orkestrasyon İhtiyacı

Container Dünyasında Yaşanan Zorluklar

- **Manuel Container Yönetimi** → Binlerce container'ı nasıl yöneteceğiz?
- **Ölçeklenebilirlik** → Trafik artışında otomatik scaling
- **Yüksek Erişilebilirlik** → Container crash olduğunda ne yapacağız?
- **Load Balancing** → Trafiği container'lar arasında dağıtma
- **Service Discovery** → Microservices birbirini nasıl bulacak?
- **Configuration Management** → Farklı ortamlarda config yönetimi
- **Rolling Updates** → Zero-downtime deployment nasıl yapılacak?

🚀 Kubernetes'in Bu Sorunlara Çözümü

> **Kubernetes (K8s)**, container'ları production ortamında çalıştırmak için Google tarafından geliştirilen açık kaynak orchestration platformudur.

Kubernetes Getirdiği Çözümler:

- ✅ **Otomatik Deployment** ve container yönetimi
- ✅ **Self-Healing** - Arızalı container'ları otomatik yeniden başlatma
- ✅ **Horizontal Scaling** - İş yüküne göre otomatik ölçekleme
- ✅ **Service Discovery** ve Load Balancing
- ✅ **Rolling Updates** ve Rollbacks
- ✅ **Secret ve Configuration** yönetimi
- ✅ **Multi-Cloud** ve Hybrid Cloud desteği

📈 Kubernetes'in Tarihçesi ve Doğuşu

```
2003-2014: Google'da Borg sistemi (internal)
     2014: Kubernetes açık kaynak olarak duyuruldu
     2015: v1.0 release - CNCF'e bağışlandı
2016-2018: Enterprise adoption başladı
2018-2020: Cloud Native ecosystem olgunlaştı
2020-2024: Industry standard haline geldi
```

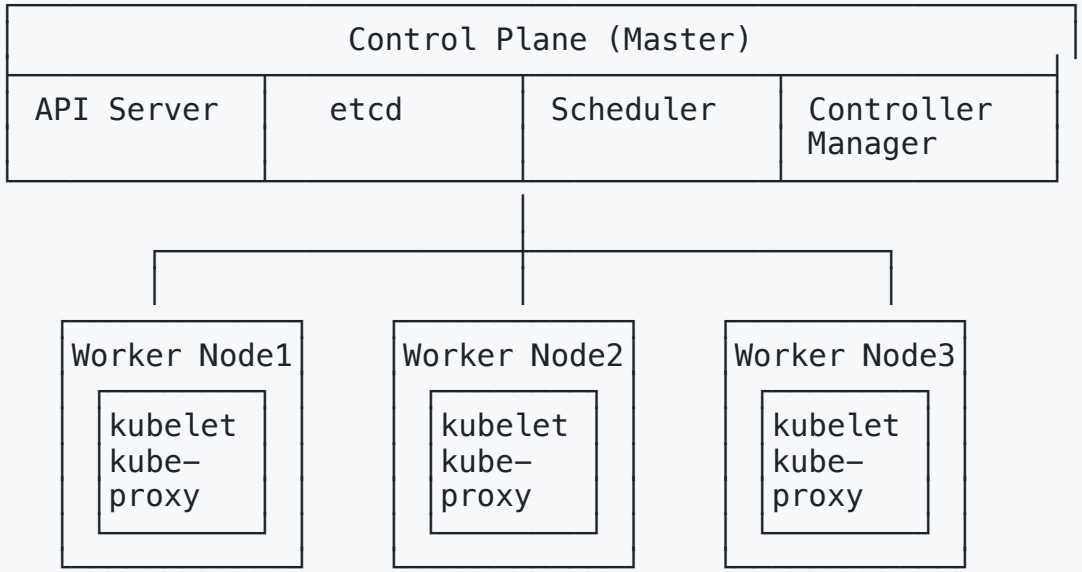
Google Borg'dan Öğrenilenler:

- **Large-scale** container orchestration
- **Resource efficiency** ve isolation
- **Declarative configuration**
- **Self-healing** sistemler

💡 **İlginç Bilgi:** Kubernetes Yunanca "kaptan/dümenchi" anlamına gelir ⚓

🏗️ Modül 1.2: Kubernetes Mimarisi ve Bileşenleri

Master ve Worker Node'lar



Control Plane Bileşenleri

API Server (kube-apiserver)

- Kubernetes'in kalbi - tüm istekler buradan geçer
- RESTful API sağlar (kubectl ile iletişim)
- Authentication ve Authorization kontrolü

etcd

- Distributed key-value store (cluster'ın hafızası)
- Cluster state'ini tutar (desired state vs actual state)

Scheduler (kube-scheduler)

- Pod'ları node'lara yerleştirir
- Resource requirements ve constraints değerlendirir

Controller Manager

- Desired state vs current state kontrolü yapar
- ReplicaSet, Deployment, Service controller'ları içerir

Worker Node Bileşenleri

kubelet

- Node'daki "agent" - her worker node'da çalışır
- Pod lifecycle yönetimi (create, update, delete)
- Control Plane ile iletişim kurar
- Container runtime ile konuşur

kube-proxy

- Network proxy ve load balancer
- Service'leri Pod'lara yönlendirir
- iptables veya IPVS kullanır

Container Runtime

- Docker, containerd, CRI-O (Container Runtime Interface)
- Container'ları çalıştırır

🌐 Modül 1.3: Kubernetes Kaynakları ve API Konseptleri

Deklaratif Yaklaşım

```
# Imperative (Nasıl yapılacağını söyler)
kubectl run nginx --image=nginx
kubectl scale --replicas=3 deployment/nginx

# Declarative (Ne istediğinizi söyler)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 3
  template:
    spec:
      containers:
      - name: nginx
        image: nginx
```

Kubernetes deklaratif yaklaşımı destekler! 🎯

🔗 API ve Kaynak Yönetimi Prensipleri

Kubernetes API

- **REST-based** API (GET, POST, PUT, DELETE)
- **Resource-based** - her kaynak bir API objesi
- **YAML** veya **JSON** formatında tanımlanır

API Groups ve Versions

```yaml
apiVersion: apps/v1        # API version
kind: Deployment           # Resource type
metadata:                  # Resource metadata
  name: my-app
  labels:
    app: web-server
spec:                      # Desired state
  replicas: 3
status:                    # Actual state (K8s tarafından yönetilir)
  readyReplicas: 2
```

🎯 Modül 1.4: Temel Kubernetes Kaynakları

Pod - En Küçük Deployment Birimi

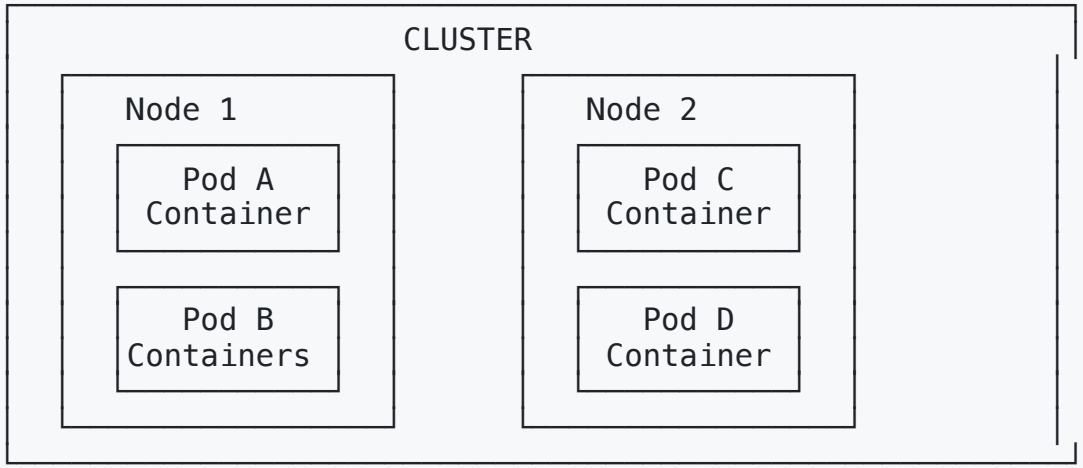
```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
  - name: web-server
    image: nginx:latest
    ports:
    - containerPort: 80
```

- **1 veya daha fazla container** içerebilir
- **Shared network** (aynı IP address)
- **Shared storage** (volumes)
- **Ephemeral** - geçici yapı, restart'ta yeniden oluşur

📦 Temel Kubernetes Kaynakları

Kaynak	Açıklama	Kullanım Senaryosu
Pod	En küçük deployment birimi	Container grupları çalıştırma
Deployment	Pod lifecycle yönetimi	Rolling updates, scaling
Service	Network abstraction	Pod'lara stable erişim
ConfigMap	Configuration data	Uygulama konfigürasyonu
Secret	Sensitive data	Passwords, certificates
Volume	Storage abstraction	Data persistence
Namespace	Resource isolation	Multi-tenant environments

🏗️ Pod, Node ve Cluster İlişkisi



Cluster = Control Plane + Worker Node(s) + Networking

🌐 Kubernetes Ekosistemi

CNCF (Cloud Native Computing Foundation)

- **Core**: Kubernetes, containerd, CoreDNS
- **Observability**: Prometheus, Grafana, Jaeger
- **Service Mesh**: Istio, Linkerd
- **Packaging**: Helm, Kustomize
- **CI/CD**: Flux, Argo

Managed Kubernetes Services

- **GKE** (Google Cloud)
- **EKS** (AWS)
- **AKS** (Azure)
- **DigitalOcean Kubernetes**

🔄 Kubernetes vs Diğer Orchestrator'lar

Özellik	Kubernetes	Docker Swarm	Nomad
Karmaşıklık	Yüksek	Düşük	Orta
Learning Curve	Steep	Kolay	Orta
Ecosystem	Çok zengin	Sınırlı	Gelişiyor
Enterprise Desteği	Mükemmel	Temel	İyi
Community	Çok aktif	Az	Aktif
Market Payı	%90+	%5	%5

Kubernetes açık ara industry leader! 🏆

Sonraki Oturum

Oturum 2: Kubernetes Ortamının Kurulumu ve İlk Adımlar

- Yerel geliştirme ortamları
- kubectl CLI aracı ve temel komutlar
- İlk uygulamanızı Kubernetes'e dağıtmak
- Kubernetes Dashboard ve görsel arayüzler

Teşekkürler!

İletişim:

- GitHub: @huseyinbabal
- LinkedIn: Hüseyin Babal